# Cahier des charges – CRM Hadithi Publishers

*Document destiné à Lovable. À fournir avec le fichier `hadithi.sql` et la charte graphique.*

## 1. Contexte et objectif

Hadithi Publishers est une maison d'édition dont le site vitrine (React + Tailwind) est déjà terminé. Le site lit ses données depuis une API Express/MySQL existante (collections, livres, auteurs, événements). Les formulaires du site (envoi de manuscrit, Book Club, Hadithi Academy, contact) ne sont pas encore branchés.

**Objectif** : créer un **back-office CRM** privé, réservé à l'équipe, qui permet de :

1. recevoir et suivre toutes les demandes venant du site (manuscrits, inscriptions, contacts) ;
2. gérer le contenu publié sur le site (livres, auteurs, collections, événements) sans toucher à la base ;
3. suivre l'activité grâce à un tableau de bord simple.

## 2. Contraintes techniques (à respecter impérativement)

- **Ne pas utiliser Supabase ni créer une autre base.** La base est **MySQL (MariaDB 10.4)**, déjà en place, accessible via l'API **Express** hébergée sur Vercel. Le CRM est un front qui consomme cette API.
- Front : **React + TypeScript + Tailwind**, appels HTTP avec **axios** via un client unique ( `src/api/client.js` ), `baseURL` lue dans une variable d'environnement ( `VITE_API_URL` ).
- Les données de référence sont chargées au démarrage dans un **store Redux** (fichiers en JS : `src/store/store.js` ), puis rafraîchies après chaque création, modification ou suppression.
- Authentification par **JWT** (connexion email + mot de passe, mots de passe hachés avec bcrypt côté API). Le token est stocké en mémoire ou cookie `httpOnly` , jamais dans le code.
- Les routes d'administration sont préfixées `/api/admin/*` et protégées par un middleware qui vérifie le token. Les routes publiques existantes ( `GET /api/collections` , `/api/books` , `/api/authors` , `/api/events` ) **ne doivent pas être modifiées**.
- Interface **responsive** (tablette et mobile utilisables), en **français**.
- Aucune clé secrète dans le front.

# 3. Identité visuelle

Reprendre la charte du site vitrine :

- **Midnight Navy** `#05182E` : couleur principale (menu latéral, boutons principaux, titres)
- **Crème** `#F6F1EA` : fonds de pages
- **Gris chaud** `#DBD7D0` : bordures, séparateurs
- **Bleu clair** `#B1BFCD` : accents, badges secondaires
- Titres en **Ariata Display**, sous-titres en **Gilroy ExtraBold**, texte courant en sans-serif lisible.
- Style sobre et éditorial : coins peu arrondis ( `rounded-sm` ), bordures fines, beaucoup d'espace, pas d'ombres lourdes.
- Champs de formulaire **soulignés** (bordure basse uniquement), labels en `text-[15px] font-semibold` , comme les formulaires du site.

# 4. Accès au CRM

Le CRM n'a **qu'un seul compte administrateur**, qui a tous les droits : lecture, création, modification, suppression et exports. Il n'y a ni rôles, ni gestion d'utilisateurs.

Le compte administrateur est créé par un script de seed, avec l'email et le mot de passe fournis en variables d'environnement. Pas d'inscription publique au CRM.

# 5. Modules fonctionnels

## 5.1 Authentification

- Page de connexion (email, mot de passe), message d'erreur clair, déconnexion automatique après 8 h d'inactivité.
- Mot de passe oublié (envoi d'un lien par email) : **phase 2**.
- Changement de son propre mot de passe depuis le profil.

## 5.2 Tableau de bord

Cartes de synthèse :

- nombre de manuscrits **à traiter**, **en lecture**, **en retard** (au-delà de 10 semaines sans réponse) ;
- nouvelles demandes des 7 derniers jours (tous types confondus) ;
- prochains événements (3 prochains, avec le nombre d'inscrits) ;
- inscrits Book Club et demandes Academy du mois.

Un fil « dernières demandes reçues » (10 lignes, lien vers la fiche).

### 5.3 Contacts (module central)

Une **base unique de contacts**, dédoublonnée par email. Chaque soumission d'un formulaire du site crée ou met à jour un contact, puis ajoute une entrée dans son historique.

- Liste avec recherche (nom, email), filtres (source, statut, tag) et pagination.
- Fiche contact : coordonnées, sources, tags, notes internes, **chronologie** de toutes les interactions (manuscrit envoyé, inscription au Book Club, demande Academy, message de contact, inscription à un événement).
- Actions : modifier, ajouter une note, ajouter ou retirer un tag, fusionner deux contacts, **exporter en CSV** (selon les filtres actifs).
- Respect du consentement : champ `consentement_at` renseigné lors de la soumission ; un contact peut être marqué « désinscrit ».

### 5.4 Manuscrits (pipeline de suivi)

Alimenté par le formulaire « Envoyer mon manuscrit » du site.

**Champs reçus** : nom et prénom, email, téléphone, titre du manuscrit, genre (*Essai*, *Mémoires*, *Récit politique*, *Histoire de vie*), résumé (300 mots max), fichier (PDF ou Word), présentation de l'auteur (200 mots), consentement et exclusivité de lecture.

**Statuts** : `reçu` → `accusé envoyé` → `en lecture` → `en comité` → `accepté` / `refusé` / `à retravailler`.

- Vue **Kanban** (une colonne par statut, glisser-déposer) et vue **tableau**, avec bascule entre les deux.
- Fiche manuscrit : toutes les informations, téléchargement du fichier, **notes de lecture**, historique des changements de statut, réponse motivée.
- **Engagement du site : réponse motivée sous 8 à 10 semaines.** Calculer une date limite à partir de la date de réception et afficher un badge de couleur : normal, à surveiller (après 8 semaines), en retard (après 10 semaines).
- Un manuscrit accepté peut être **converti en livre** : création d'un brouillon dans le catalogue ( `is_published = 0` ) avec titre, genre et auteur pré-remplis.

### 5.5 Book Club et Hadithi Academy

- **Book Club** (gratuit) : nom, email, collection préférée (*Toutes*, *Legacy*, *Visions du Monde*, *Kids*). Liste des inscrits, filtre par collection, export CSV pour la lettre mensuelle.

- **Academy** : nom, email, atelier choisi, projet en deux lignes (300 caractères max). Statuts : `nouvelle` → `contactée` → `inscrite` / `refusée`. Les ateliers sont les suivants : *Structurer un récit de vie*, *Passer du témoignage oral au manuscrit*, *Construire un essai à partir d'une expérience de terrain*.
- Les deux listes sont gérables depuis un même écran à deux onglets.

### 5.6 Messages de contact

Boîte de réception des messages envoyés depuis la page contact : statuts `nouveau` / `en cours` / `traité`, notes internes, réponse envoyée par email depuis le CRM (phase 2) ou marquée comme traitée manuellement.

### 5.7 Événements

Gestion complète de la table `events` existante : titre, slug (généré automatiquement et modifiable), date, ville, lieu, type (*Lancement*, *Salon*, *Conférence*, *Dédicace*), description, informations pratiques, **image** ( `img_url` ), publication.

- Association à un ou plusieurs **auteurs** et **livres** (tables `event_authors` et `event_books` ), via des sélecteurs multiples.
- **Inscriptions** : les événements actuels demandent de s'inscrire par email. Le CRM gère ces inscriptions dans une table dédiée, avec liste des inscrits par événement, jauge de capacité optionnelle, export CSV et feuille d'émargement imprimable.
- Distinction visuelle entre événements à venir et archives (date passée).

### 5.8 Catalogue

Gestion des tables existantes, avec formulaires de création et d'édition.

- **Livres** ( `books` ) : titre, slug, auteur, collection, format (*Essai*, *Mémoires & Trajectoires*, *Cahiers*, *Hors-série*), thème (*Économie & développement*, *Gouvernance & institutions*, *Relations internationales*, *Mémoires*, *Jeunesse*, *Récit*), disponibilité (*Nouveauté*, *Catalogue*, *À paraître*), prix (saisi en euros, stocké en centimes dans `price_cents` ), pages, ISBN, date de parution, tonalité de couverture (*navy*, *warm*, *blue*, *cream*), couverture, extrait PDF, histoire, pourquoi le lire, publication.
- **Auteurs** ( `authors` ) : nom, slug, photo, rôle, biographie, liens (JSON : liste de `{label, href}` , éditeur de liens dynamique), publication.
- **Collections** ( `collections` ) : nom, slogan, description, badge, couleur, texte alternatif, couverture, ordre d'affichage (réordonnable par glisser-déposer).
- Les listes déroulantes des champs `enum` sont **lues depuis la base** ou définies en constantes identiques à la base, jamais saisies librement.
- Bascule rapide **publié / brouillon** dans les listes ( `is_published` ).

- Téléversement d'images et de PDF (voir section 8). Prévisualisation avant publication.
- Suppression : confirmation obligatoire, et blocage avec message explicite si l'élément est lié à d'autres (un auteur qui a des livres, une collection qui a des livres).

### 5.9 Journal d'activité

Trace de chaque action (qui, quoi, quand, sur quel élément) : création, modification, suppression, changement de statut, export. Consultable par l'administrateur, avec filtres.

## 6. Modèle de données

### 6.1 Tables existantes (ne pas modifier leur structure sans validation)

`authors`, `books`, `collections`, `events`, `event_authors`, `event_books`. Les clés primaires sont des **slugs** (`varchar`) et non des identifiants numériques ; les relations passent par `author_slug`, `collection_id`, `event_slug`, `book_slug`. Le fichier `hadithi.sql` fait foi.

### 6.2 Nouvelles tables à créer

```
-- Une seule ligne attendue : le compte administrateur
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(190) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  last_login_at DATETIME NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE contacts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  email VARCHAR(190) NOT NULL UNIQUE,
  phone VARCHAR(50) NULL,
  tags JSON NULL,
  notes TEXT NULL,
  consent_at DATETIME NULL,
  unsubscribed TINYINT(1) NOT NULL DEFAULT 0,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE manuscripts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  contact_id INT NOT NULL,
```

```
  title VARCHAR(255) NOT NULL,
  genre ENUM('Essai','Mémoires','Récit politique','Histoire de vie') NOT
NULL,
  summary TEXT NOT NULL,
  author_bio TEXT NOT NULL,
  file_url VARCHAR(500) NULL,
  status
ENUM('recu','accuse_envoye','en_lecture','en_comite','accepte','refuse','a_re
travailler') NOT NULL DEFAULT 'recu',
  reading_notes TEXT NULL,
  decision_note TEXT NULL,
  received_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  answered_at DATETIME NULL,
  FOREIGN KEY (contact_id) REFERENCES contacts(id)
);

CREATE TABLE book_club_members (
  id INT AUTO_INCREMENT PRIMARY KEY,
  contact_id INT NOT NULL,
  collection_id VARCHAR(20) NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (contact_id) REFERENCES contacts(id),
  FOREIGN KEY (collection_id) REFERENCES collections(id) ON DELETE SET NULL
);

CREATE TABLE academy_requests (
  id INT AUTO_INCREMENT PRIMARY KEY,
  contact_id INT NOT NULL,
  workshop VARCHAR(255) NOT NULL,
  project VARCHAR(300) NOT NULL,
  status ENUM('nouvelle','contactee','inscrite','refusee') NOT NULL DEFAULT
'nouvelle',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (contact_id) REFERENCES contacts(id)
);

CREATE TABLE contact_messages (
  id INT AUTO_INCREMENT PRIMARY KEY,
  contact_id INT NOT NULL,
  subject VARCHAR(255) NULL,
  message TEXT NOT NULL,
  status ENUM('nouveau','en_cours','traite') NOT NULL DEFAULT 'nouveau',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (contact_id) REFERENCES contacts(id)
);
```

```
CREATE TABLE event_registrations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  event_slug VARCHAR(100) NOT NULL,
  contact_id INT NOT NULL,
  seats INT NOT NULL DEFAULT 1,
  attended TINYINT(1) NOT NULL DEFAULT 0,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_event_contact (event_slug, contact_id),
  FOREIGN KEY (event_slug) REFERENCES events(slug) ON DELETE CASCADE,
  FOREIGN KEY (contact_id) REFERENCES contacts(id)
);

CREATE TABLE activity_log (
  id INT AUTO_INCREMENT PRIMARY KEY,
  action VARCHAR(50) NOT NULL,
  entity VARCHAR(50) NOT NULL,
  entity_id VARCHAR(100) NULL,
  details JSON NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Ajouter aussi `capacity INT NULL` à la table `events` pour la jauge d'inscriptions (migration à part, valeur nulle par défaut).

# 7. API

## 7.1 Routes publiques (appelées par le site vitrine, sans authentification)

| Méthode | Route | Rôle |
|---|---|---|
| POST | `/api/forms/manuscript` | Envoi de manuscrit ( `multipart/form-data` ) |
| POST | `/api/forms/book-club` | Inscription au Book Club |
| POST | `/api/forms/academy` | Demande d'atelier |
| POST | `/api/forms/contact` | Message de contact |
| POST | `/api/events/:slug/register` | Inscription à un événement |

Ces routes doivent être protégées contre l'abus : validation stricte des champs, **limitation de débit** par IP, champ « piège » anti-spam (honeypot), taille et type de fichier contrôlés (PDF, DOC, DOCX, 10 Mo max). Elles répondent par un JSON simple ( `{ ok: true }` ou la liste des erreurs de validation), que le front du site affiche sous chaque champ.

### 7.2 Routes d'administration (JWT obligatoire)

- `POST /api/admin/login`, `GET /api/admin/me`
- CRUD complet sous `/api/admin/` : `books`, `authors`, `collections`, `events`, `contacts`, `manuscripts`, `book-club`, `academy`, `messages`, `registrations`
- `PATCH /api/admin/manuscripts/:id/status` (change le statut et écrit dans le journal)
- `POST /api/admin/manuscripts/:id/convert` (conversion en brouillon de livre)
- `GET /api/admin/stats` (chiffres du tableau de bord)
- `GET /api/admin/export/:type` (CSV)
- `POST /api/admin/uploads` (images et PDF)
- `GET /api/admin/activity`

Pagination, tri et filtres passent par des paramètres de requête (`?page=`, `?limit=`, `?q=`, `?status=`).

## 8. Fichiers et médias

- Les images et PDF sont téléversés vers un **stockage externe** (Cloudinary ou équivalent), car Vercel ne conserve pas les fichiers. L'API ne stocke en base que l'URL.
- Formats : images WebP, JPG, PNG (5 Mo max) ; documents PDF, DOC, DOCX (10 Mo max).
- Les manuscrits reçus sont **privés** : accessibles uniquement par une URL signée générée pour un utilisateur connecté.

## 9. Sécurité et conformité

- Mots de passe hachés (bcrypt), jetons à durée limitée, verrouillage temporaire après 5 échecs de connexion.
- Requêtes SQL **paramétrées** uniquement.
- CORS limité aux domaines du site et du CRM.
- Données personnelles : consentement enregistré avec sa date, export et suppression d'un contact sur demande, mention RGPD dans les formulaires.
- Les manuscrits sont confidentiels : accès restreint, aucune diffusion de lien public.

## 10. Phasage

**Phase 1 : socle (à livrer en premier)**

1. Authentification, rôles, mise en page générale du CRM
2. Branchement des formulaires du site (routes publiques)
3. Contacts, Manuscrits, Book Club et Academy, Messages
4. Tableau de bord

**Phase 2 : contenu**

5. Gestion du catalogue (livres, auteurs, collections)
6. Gestion des événements et de leurs inscriptions
7. Téléversement de fichiers, conversion manuscrit → livre

**Phase 3 : confort**

8. Journal d'activité
9. Envoi d'emails depuis le CRM (accusé de réception, réponse motivée) avec modèles
10. Mot de passe oublié, notifications par email à l'équipe

## 11. Critères d'acceptation

- Une soumission du formulaire manuscrit sur le site apparaît dans le CRM en moins de 5 secondes, avec son fichier téléchargeable.
- Un même email qui s'inscrit au Book Club puis envoie un manuscrit ne crée **qu'un seul contact**, avec deux entrées dans sa chronologie.
- Un livre créé et publié dans le CRM apparaît sur le site sans intervention technique ; repassé en brouillon, il disparaît du site.
- Sans token valide, aucune route d'administration ne répond (erreur 401), même en appelant l'API directement.
- Les manuscrits dont la réponse dépasse 10 semaines sont signalés en rouge sur le tableau de bord.
- Toutes les pages sont utilisables sur un écran de 375 px de large.

## 12. Livrables attendus

- Code source du front CRM (React, TypeScript, Tailwind)
- Routes API Express correspondantes, avec les scripts de migration SQL de la section 6.2
- Script de seed pour le compte administrateur
- Fichier `.env.example` listant toutes les variables nécessaires
- Court guide de déploiement (Vercel pour l'API et le front)